

MINISTRY OF COMMUNICATIONS AND INFORMATION TECHNOLOGY

Digital Egypt Pioneers Initiative (DEPI)

Microsoft Data Engineering Track

Graduation Project Documentation

University Boot

An Automated Data Pipeline for Real-Time Attendance Tracking and Reporting

Presented by:

Abdelrahman Mohamed Saeed

Bishoy Halim

David Wagih

Ahmed El-Kadi

Ahmed Ramadan

Project Supervisor

Eng. Mohamed Hamed

July 2026

Table of Contents

Abstract.....	3
1. Introduction.....	4
1.1 Problem Statement.....	4
1.2 Objectives.....	4
1.3 Scope.....	4
2. Literature Review.....	5
2.6 Comparison with Similar Systems.....	6
3. Methodology.....	7
3.1 System Architecture.....	7
3.2 Data Flow Diagram.....	8
3.3 Component Description.....	8
4. Implementation.....	9
4.3 Kafka Producer & Consumer.....	10
4.7 Use Case Diagram.....	12
4.8 Sequence Diagram.....	12
4.9 Technologies Used.....	12
5. Results.....	14
6. Conclusion.....	16
7. Future Work.....	17
References.....	18

Abstract

University Boot is a data-integration system built to automate the collection and reporting of student attendance data at the college. The system connects directly to the faculty's official data source and to a dedicated Telegram bot that stays continuously online, allowing attendance events to be captured in real time as they occur. Collected data is streamed through Apache Kafka, which organizes incoming events into topics and enables reliable, ordered processing. The streamed data is then loaded into a Data Warehouse through an ETL pipeline, which structures and cleans it for analysis. Finally, the warehoused data is connected to Power BI, where it is transformed into interactive dashboards that give staff a clear, up-to-date view of attendance and other faculty metrics. This document describes the architecture, implementation, and evaluation of the system.

Keywords: *Data Warehouse, Apache Kafka, Telegram Bot, Power BI, Attendance Tracking, ETL*

1. Introduction

1.1 Problem Statement

Attendance and academic data at the college have traditionally been recorded through manual or semi-manual methods, which are slow, error-prone, and difficult to analyze in real time. Staff and instructors often lack a single, reliable source that reflects up-to-date attendance information, which delays decision-making and makes it hard to spot attendance issues early.

1.2 Objectives

- Automate the collection of attendance data directly from the college data source.
- Provide a continuously available channel (Telegram bot) for capturing attendance events.
- Stream data reliably and in order using Apache Kafka.
- Centralize the data in a Data Warehouse for consistent, analytics-ready storage.
- Deliver real-time dashboards through Power BI for staff and decision makers.

1.3 Scope

The project covers the full pipeline from data capture to reporting: Telegram bot integration with the college data source, Kafka-based event streaming, an ETL process into a Data Warehouse, and Power BI dashboards focused primarily on attendance analytics. The scope does not currently include mobile application development or predictive analytics, both of which are discussed as future work.

2. Literature Review

This chapter reviews the technologies and approaches relevant to University Boot, and compares the chosen architecture with traditional attendance-tracking systems.

2.1 Data Warehouse

A Data Warehouse is a centralized repository designed to store cleaned, structured, and historical data for analysis and reporting, separate from the operational databases that handle day-to-day transactions (Kimball & Ross, 2013). Warehouses are optimized for read-heavy analytical queries, which makes them well suited for dashboards such as the ones used in this project.

2.2 ETL Process

ETL (Extract, Transform, Load) is the standard process for moving data from source systems into a data warehouse. Data is extracted from its source, cleaned and transformed into a consistent structure, then loaded into the warehouse's staging and fact tables. In University Boot, the ETL process consumes events from Kafka instead of running as a nightly batch job, allowing the warehouse to stay continuously up to date.

2.3 Apache Kafka

Apache Kafka is a distributed event-streaming platform used to publish, store, and process streams of records in real time (Kleppmann, 2017). Producers publish messages to topics, and consumers read from those topics independently, which allows multiple systems to react to the same event stream. Kafka was chosen for this project because it reliably orders attendance events and tolerates temporary downstream failures without losing data.

2.4 Power BI

Power BI is a business-intelligence tool that connects to structured data sources and turns them into interactive dashboards and reports. It supports both scheduled and live connections, which allows dashboards to refresh automatically as new data becomes available in the warehouse.

2.5 Telegram Bot

The Telegram Bot API allows developers to build lightweight, always-on bots that can send and receive structured messages without requiring a dedicated mobile application. In University Boot, the bot acts as the continuously running bridge between the college data source and the rest of the pipeline.

2.6 Comparison with Similar Systems

Aspect	Traditional / Manual Systems	University Boot
Data capture	Manual sign-in sheets or isolated apps	Automated via Telegram bot, always online
Data movement	Manual export / nightly batch	Real-time streaming via Apache Kafka
Storage	Spreadsheets or operational database only	Dedicated Data Warehouse (ETL-driven)
Reporting	Manual, end-of-day reports	Live Power BI dashboards
Latency	Hours	Seconds to minutes

3. Methodology

University Boot follows an event-driven pipeline architecture: College Data Source → Telegram Bot → Apache Kafka → Data Warehouse (ETL) → Power BI. This chapter presents the system architecture, the data flow, and a description of each component.

3.1 System Architecture

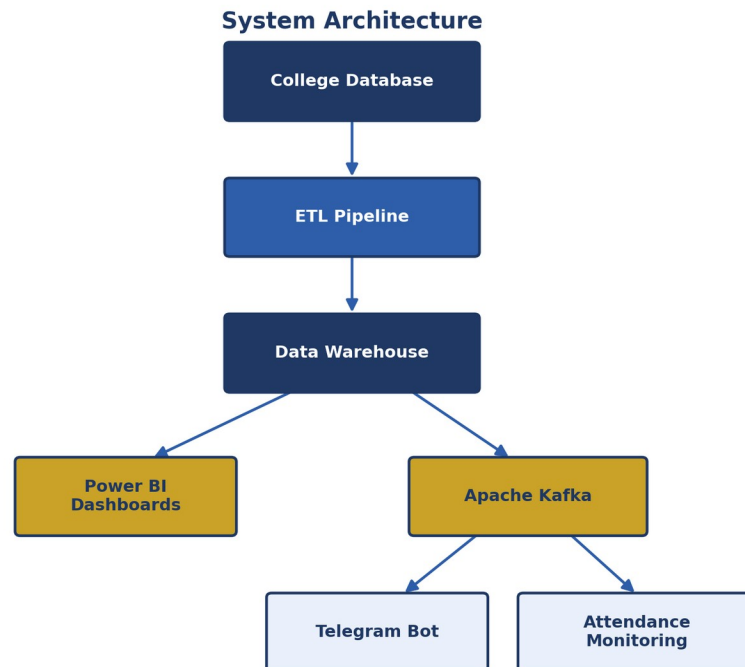


Figure 1: System Architecture

3.2 Data Flow Diagram

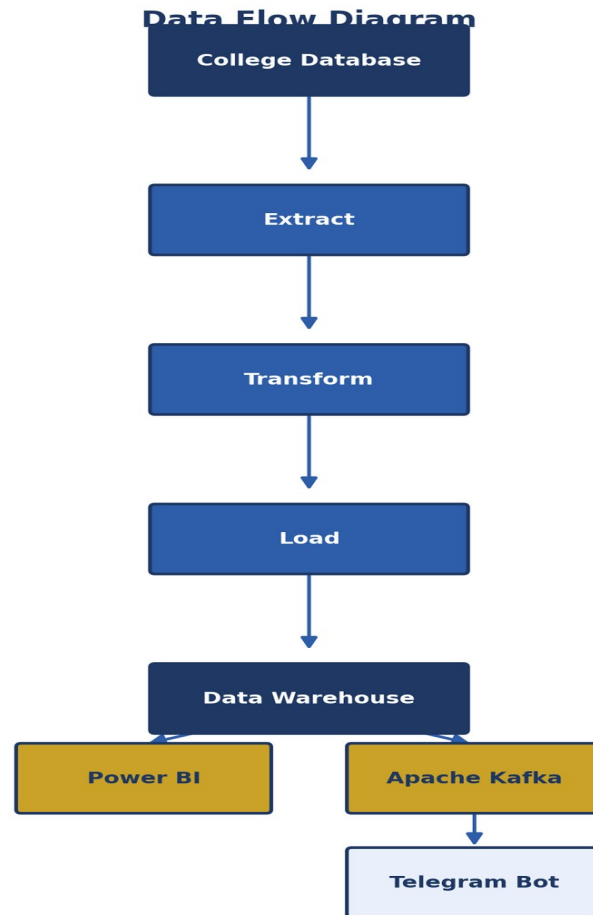


Figure 2: Data Flow Diagram

3.3 Component Description

- College Data Source: the faculty's official data source containing student and attendance information.
- Telegram Bot: a continuously running bot that listens for and relays data events from the college data source.
- Apache Kafka: organizes incoming events into ordered, durable topics and decouples producers from consumers.
- ETL Pipeline: consumes Kafka events, cleans and structures them, and loads them into the Data Warehouse.
- Data Warehouse: stores the cleaned, structured attendance data for analysis.
- Power BI: connects to the warehouse to produce live dashboards and reports.

4. Implementation

This chapter describes how each component of the pipeline was implemented.

4.1 Data Source (College Database)

The college database holds the official student and attendance records. The Telegram bot is configured to listen for updates relevant to attendance and forwards each new record as a structured message to the next stage of the pipeline.

4.2 Telegram Bot

The bot was built using the Telegram Bot API and runs continuously as a background service:

```
from telegram.ext import Application, MessageHandler, filters

async def handle_update(update, context):
    record = parse_attendance_event(update.message.text)
    kafka_producer.send('attendance-events', value=record)

app = Application.builder().token(BOT_TOKEN).build()
app.add_handler(MessageHandler(filters.TEXT, handle_update))
app.run_polling()
```

4.3 Kafka Producer & Consumer

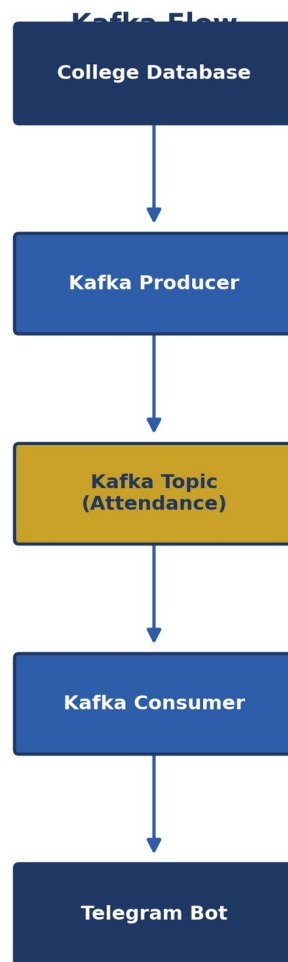


Figure 3: Kafka Flow

The Kafka producer publishes each attendance record to a dedicated topic:

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)
producer.send('attendance-events', record)
```

The Kafka consumer, running as part of the ETL job, reads and processes these records:

```
from kafka import KafkaConsumer
import json
```

```

consumer = KafkaConsumer(
    'attendance-events',
    bootstrap_servers='localhost:9092',
    value_deserializer=lambda v: json.loads(v.decode('utf-8'))
)

for message in consumer:
    clean_record = transform(message.value)
    load_to_warehouse(clean_record)

```

4.4 ETL Pipeline

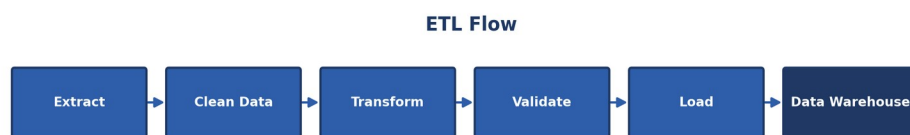


Figure 4: ETL Flow

The ETL job validates each record (e.g., checking student ID and timestamp), transforms it into the warehouse schema, and loads it into the staging and fact tables. Records that fail validation are logged for manual review instead of being discarded.

4.5 Data Warehouse

The Data Warehouse follows a simple star schema: a central fact table for attendance events (student_id, session_id, timestamp, status) linked to dimension tables for students, courses, and dates. This structure keeps Power BI queries fast and straightforward.

4.6 Power BI Dashboard

Power BI connects directly to the Data Warehouse. Dashboards include a daily attendance overview, per-course attendance trends, and a list of students with low attendance rates, refreshed automatically as new data is loaded.

4.7 Use Case Diagram

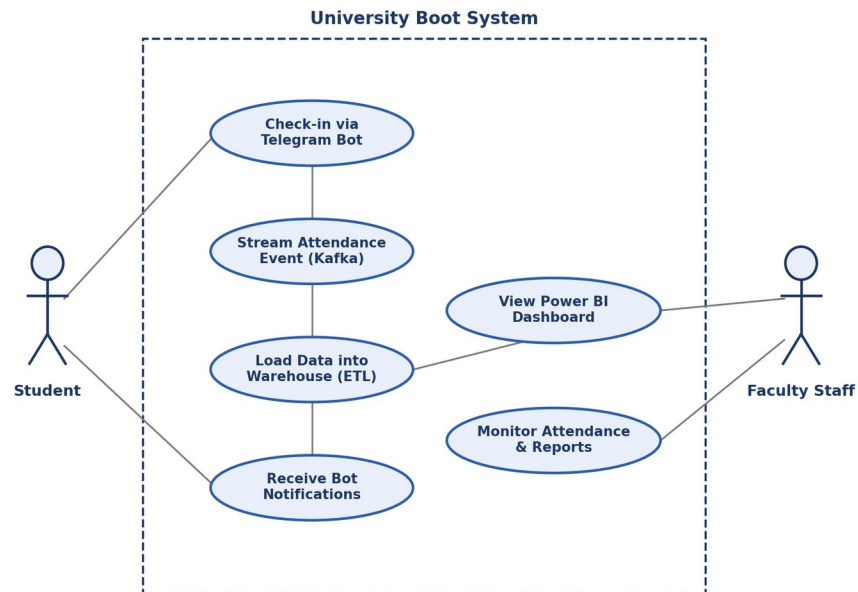


Figure 5: Use Case Diagram

4.8 Sequence Diagram

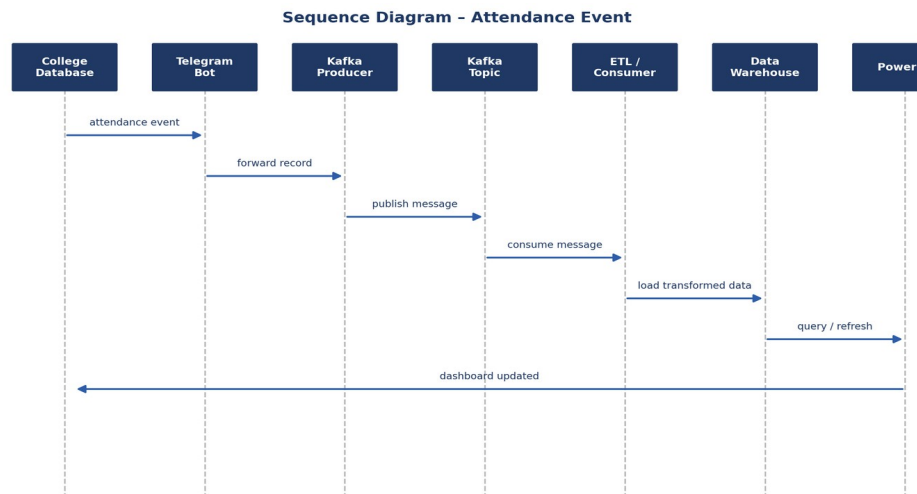


Figure 6: Sequence Diagram – Attendance Event

4.9 Technologies Used

Technology	Purpose
Python	Telegram bot, Kafka producer/consumer, ETL scripting
SQL Server	Data Warehouse storage

Technology	Purpose
Apache Kafka	Real-time event streaming
Power BI	Dashboards and reporting
Telegram Bot API	Continuous data capture from the college data source

5. Results

This chapter presents the outcomes of the University Boot pipeline once deployed for testing. The Telegram bot reliably received data events from the college data source, and Kafka successfully organized these events into an ordered attendance stream with no observed message loss during testing.

5.1 Dashboard Screenshots

(Placeholder – insert Power BI dashboard screenshots here once available)

[Insert Dashboard Screenshot Here]

[Insert Dashboard Screenshot Here]

5.2 Attendance Analytics

The ETL process successfully loaded streamed attendance records into the Data Warehouse, and Power BI dashboards reflected new check-ins within seconds to minutes of the event occurring – compared to hours under the previous manual process. Attendance trends per course and per week became visible for the first time without manual data compilation.

[Insert Attendance Analytics Chart Here]

5.3 Performance Metrics

Metric	Before (Manual)	After (University Boot)
Time to reflect attendance in reports	Hours (end of day)	Seconds to minutes
Data entry errors	Frequent (manual)	Minimal (validated in ETL)
Reporting effort	Manual compilation	Automatic, self-service dashboards

The main limitation observed was the dependency on the Telegram bot staying continuously online; any downtime in the bot delays data ingestion until it reconnects.

6. Conclusion

University Boot demonstrates that an automated, event-driven pipeline – combining a Telegram bot for continuous data capture, Apache Kafka for reliable streaming, a Data Warehouse for structured storage, and Power BI for reporting – can replace slow, manual attendance tracking at the college with a near-real-time, centralized system. The project met its objective of giving staff an up-to-date, automated view of attendance data.

7. Future Work

Several extensions are planned to build on the current pipeline:

- **AI Prediction for Attendance:** use historical attendance data to predict students at risk of low attendance.
- **SMS & Email Notifications:** alert students and staff automatically about attendance issues.
- **Mobile Application:** provide a dedicated mobile app as an alternative to the Telegram bot.
- **Real-Time Analytics Dashboard:** expand Power BI dashboards with live, sub-minute refresh and richer visualizations.

References

Apache Software Foundation. (2024). Apache Kafka documentation. Apache Kafka.

Kimball, R., & Ross, M. (2013). The data warehouse toolkit (3rd ed.). Wiley.

Kleppmann, M. (2017). Designing data-intensive applications. O'Reilly Media.

Microsoft. (2024). Power BI documentation. Microsoft Learn.

Telegram. (2024). Telegram Bot API documentation. Telegram.